

# YouTube Content Creation Analysis

## PROJECT OBJECTIVE

To analyze YouTube channels across multiple content categories to identify the factors that influence video performance, audience engagement, and channel growth, enabling content creators to make data-driven decisions regarding content strategy and publishing schedules

## BUSINESS QUESTIONS

- 1) Which content categories get the most engagement (using engagement rate (likes per view, comments per view, not raw views) and which are over-saturated (lot of supply, little engagement) and under-saturated categories (little supply, strong reward)
- 2) Which channels lead on subscriber and views – and what do the strongest performers appear to do differently?
- 3) Does video length relate to engagement?
- 4) Which video characteristics (description length, tag count, duration, and title features where applicable) are associated with higher-performing videos?
- 5) Does upload timing – day of week and hour of publish – relate to engagement?
- 6) How have channel creation, video uploads, and subscriber growth changed over time?
- 7) Within individual channels, what distinguishes the top performing videos from weak ones?

## STAKEHOLDERS FOR THIS PROJECT

- 1) YouTube Content Creators

## CHANNELS CONTENT TYPE COVERED

- 1) Comedy
- 2) Education
- 3) Entertainment
- 4) Gaming
- 5) How-to & Style
- 6) Music
- 7) News & Politics
- 8) People & Blogs
- 9) Science & Technology

## DATA GENERATION

### STEPS INVOLVED FOR CREATION OF CHANNELS CSV FILE FROM API

**Endpoint** = <https://www.googleapis.com/youtube/v3/channels>

#### **Columns:**

`channel_id` -> The ID that YouTube uses to uniquely identify the channel.

`channel_title` -> The channel's title.

`channel_description` -> The channel's description. The property's value has a maximum length of 1000 characters.

`channel_creation_date` -> The date and time that the channel was created. The value is specified in [ISO 8601](#) format.

`channel_country` -> The country with which the channel is associated.

`thumbnail` -> The default thumbnail image

`subscriberCount` -> The number of subscribers that the channel has. This value is rounded down to three significant figures

`totalViewCount` -> The sum of the number of times all the videos in all formats have been viewed for a channel.

`totalVideoCount` -> The number of public videos uploaded to the channel. Note that the value reflects the count of the channel's public videos only, even to owners.

#### **Step 1: Retrieve Channel IDs using the Search API**

The YouTube **Channels** endpoint requires one or more channel IDs (`id` parameter) to return channel details. Since the IDs were not known beforehand, the project first queried the **Search** endpoint.

Search terms covering multiple content categories were used:

- Technology
- Education
- Gaming
- Music
- News
- Finance
- Sports
- Cooking
- Travel
- Fitness

The Search API returned a list of matching videos/channels.

```
BASE_URL = "https://www.googleapis.com/youtube/v3"
```

```
endpoint="search"
search_url=f"{BASE_URL}/{endpoint}"
params_search = {
    "key": API_KEY,
    "part": "snippet",
    "q" : "technology|education|gaming|music|news|finance|sport|cooking|travel|fitness",
    "maxResults" : 50
}
```

```
# Gather all channel ids
all_channel_ids = set()
while True:
    response=requests.get(search_url,params=params_search)
    data=response.json()
    channel_ids = list(set(channels['snippet']['channelId'] for channels in data['items']))
    all_channel_ids.update(channel_ids)
    if "nextPageToken" not in data:
        break
    params_search={
        "key": API_KEY,
        "part": "snippet",
        "q" : "technology|education|gaming|music|news|finance|sport|cooking|travel|fitness",
        "maxResults" : 50,
        "pageToken": data['nextPageToken']
    }
```

## Step 2: Fetch Channel Details

After collecting the unique channel IDs, the **Channels** endpoint was called to retrieve detailed metadata for each channel.

Since the YouTube Data API accepts a maximum of **50 channel IDs per request**, the IDs were processed in batches.

The following information was collected for each channel:

- Channel ID
- Channel Name
- Description
- Country
- Creation Date
- Thumbnail
- Subscriber Count
- Video Count
- View Count

```
# Gather all channels information using channel ids retrieved above
endpoint="channels"
channel_url=f"{BASE_URL}/{endpoint}"

all_channel_data = []

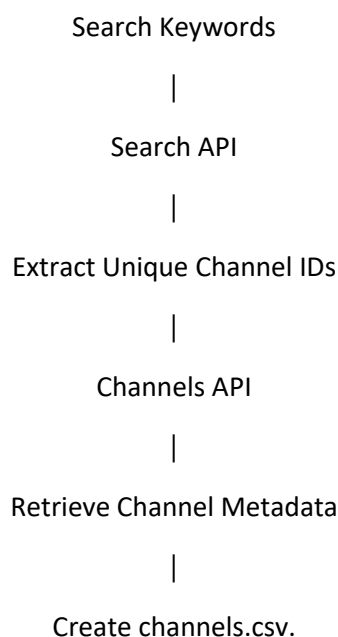
# Fetch channel details in batches of 50, as the YouTube API accepts up to 50 channel IDs per request
for batch in range(0, len(all_channel_ids), 50):
    params_channels = {
        "key": API_KEY,
        "part": "snippet,statistics,status",
        "id": ', '.join(list(all_channel_ids)[batch:batch+50])
    }
    response=requests.get(channel_url, params=params_channels).json()
    if "items" in response:
        all_channel_data.extend(response["items"])
```

```
channels_data = pd.DataFrame({"channel_id": [data['id'] for data in all_channel_data],
                              "channel_title": [data['snippet'].get('title') for data in all_channel_data],
                              "channel_description": [data['snippet'].get('description') for data in all_channel_data],
                              "channel_creation_date": [data['snippet'].get('publishedAt') for data in all_channel_data],
                              "channel_country": [data['snippet'].get('country') for data in all_channel_data],
                              "thumbnail": [data['snippet']['thumbnails']['default']['url'] for data in all_channel_data],
                              "subscriberCount": [data['statistics'].get('subscriberCount') for data in all_channel_data],
                              "totalViewCount": [data['statistics'].get('viewCount') for data in all_channel_data],
                              "totalVideoCount": [data['statistics'].get('videoCount') for data in all_channel_data]})
```

```
channels_data.to_csv("channels.csv")
```

## Why use the Search endpoint before the Channels endpoint?

The **Channels** endpoint requires the id parameter (atleast one filter is required to send get request to this endpoint), meaning it cannot return a list of channels without passing the filter parameter. The **Search** endpoint, however, allows searching by keywords and returns matching channel IDs. Therefore, the workflow was:



## STEPS INVOLVED FOR CREATION OF SUBSCRIPTIONS CSV FILE FROM API

**Endpoint** = <https://www.googleapis.com/youtube/v3/subscriptions>

### Columns:

**channel\_id** -> The value that YouTube uses to uniquely identify the channel that the user subscribed to.

**subscriber\_id** -> The ID that YouTube uses to uniquely identify the subscriber's channel.

**subscription\_time** -> The date and time that the subscription was created. The value is specified in [ISO 8601](https://en.cppreference.com/w/cpp/string/basic/basic_iso8601) format.

### Step 1: Retrieve Channel IDs from channels.csv

```
BASE_URL = "https://www.googleapis.com/youtube/v3"
```

```
channels_df = pd.read_csv('channels.csv')
```

```
channel_ids = channels_df['channel_id']
channel_ids

0      UCqoUtFTzx-fcFDdZLOGwL_w
1      UCGqfnwqfmdj94kiLUJbMLw
2      UCTGqaTypkZvxwIsxQV_brFQ
3      UC6UTUmL5RbRPPRgnptAVu1g
4      UCyJ7AXuq17EBaH972y56qyg
...
448     UC2rVBfyUoUDnN1ubYkugAvQ
449     UCTmcd00T18B09eLxcFAXo1Q
450     UCfx_nR5D1g4s0qqzND5D16w
451     UC8rdIjtQWq43M_0s2dr078A
452     UCNN8haGXA-cItnqtfsN45Gg
Name: channel_id, Length: 453, dtype: str
```

### Step 2: Fetch Subscription Details

```
endpoint="subscriptions"
subscriptions_url=f"{BASE_URL}/{endpoint}"

all_subscription_details=[]

for channel_id in channel_ids:
    params_subscriptions={
        "key": API_KEY,
        "part": "snippet,contentDetails,subscriberSnippet",
        "channelId": channel_id
    }
    while True:
        response = requests.get(subscriptions_url, params=params_subscriptions)
        data = response.json()

        all_subscription_details.extend(data.get("items", []))

        if "nextPageToken" not in data:
            break

        params_subscriptions["pageToken"] = data["nextPageToken"]
```

```
subscribers_df = pd.DataFrame({"channel_id": [subscription['snippet']['resourceId']['channelId'] for subscription in all_subscription_details],
                              "subscriber_id": [subscription['snippet']['channelId'] for subscription in all_subscription_details],
                              "subscription_time": [subscription['snippet']['publishedAt'] for subscription in all_subscription_details]})
subscribers_df.to_csv('subscriptions.csv', index=False)
```

```
len(all_subscription_details)
```

```
10085
```

## STEPS INVOLVED FOR CREATION OF UPLOAD PLAYLIST CSV FILE FROM API

**Endpoint** = <https://www.googleapis.com/youtube/v3/playlistItems>

### Columns:

channel\_id -> The ID that YouTube uses to uniquely identify the channel.

upload\_playlist\_id -> The ID of the playlist that contains the channel's uploaded videos.

video\_id -> The ID that YouTube uses to uniquely identify a video.

### Step 1: Retrieve Uploads Playlist IDs

After obtaining all channel IDs, the channels API was called with the `contentDetails` part to retrieve each channel's Uploads playlist.

```
BASE_URL = "https://www.googleapis.com/youtube/v3"
```

```
channels_df= pd.read_csv('../channels.csv')
all_channel_ids = channels_df['channel_id']
len(all_channel_ids)
```

453

```
# Gather content details information using channel ids retrieved above
endpoint="channels"
channel_url=f"{BASE_URL}/{endpoint}"

content_details = []

# Fetch channel details in batches of 50, as the YouTube API accepts up to 50 channel IDs per request
for batch in range(0,len(all_channel_ids),50):
    params_channels = {
        "key": API_KEY,
        "part": "contentDetails",
        "id": ','.join(list(all_channel_ids)[batch:batch+50])
    }
    response=requests.get(channel_url,params=params_channels).json()
    if "items" in response:
        content_details.extend(response["items"])
len(content_details)
```

453

```
channel_upload_playlist = [{
    "channel_id":content['id'],
    "uploads_playlist_id": content["contentDetails"]["relatedPlaylists"].get("uploads")
}]
for content in content_details
len(channel_upload_playlist)
```

453

### Step 2: Fetch Videos from Each Uploads Playlist

Each Uploads playlist was queried using the `playlistItems` endpoint. Since the API returns a maximum of 50 videos per request, pagination was implemented using the `nextPageToken`. This ensured that all videos from every channel were retrieved, regardless of playlist size.

```

: endpoint="playlistItems"
upload_playlist_url = f"{BASE_URL}/{endpoint}"

upload_playlist_details=[]

for i in range(0,len(channel_upload_playlist)):
    params_upload_playlist = {
        "key": API_KEY,
        "part": "contentDetails,snippet",
        "playlistId": channel_upload_playlist[i]['uploads_playlist_id'],
        "maxResults": 50
    }
    while True:
        response= requests.get(upload_playlist_url, params=params_upload_playlist)
        data= response.json()
        upload_playlist_details.extend(data.get('items',[]))
        if(data.get('nextPageToken')):
            params_upload_playlist['pageToken'] = data.get('nextPageToken')
        else:
            break

```

### Step 3: Extract Required Information and Save

```

upload_playlist_df = pd.DataFrame({
    "channel_id":[detail['snippet']['channelId'] for detail in upload_playlist_details],
    "upload_playlist_id":[detail['snippet']['playlistId'] for detail in upload_playlist_details],
    "video_id":[detail['contentDetails']['videoId'] for detail in upload_playlist_details]
})

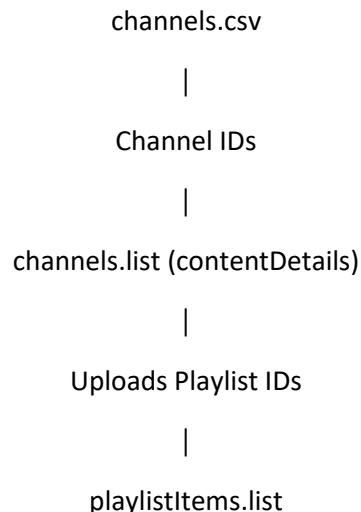
upload_playlist_df.to_csv('upload_playlist.csv')

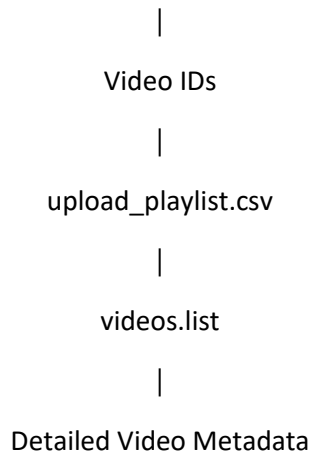
```

### Why this file needs to be created?

The purpose of this step is to create a mapping between each YouTube channel and the videos present in its Uploads playlist. Since the YouTube Data API does not provide a direct endpoint to retrieve all videos uploaded by a channel, the Uploads playlist acts as an intermediate resource for accessing a channel's videos.

The Videos API (videos.list) requires one or more video IDs as input. However, after retrieving the list of channel IDs, only channel-level information is available. There is no direct way to pass a channel ID to the Videos API and obtain all uploaded videos.





## STEPS INVOLVED FOR CREATION OF VIDEOS CSV FILE FROM API

**Endpoint** = <https://www.googleapis.com/youtube/v3/videos>

### Columns:

**video\_id** -> The ID that YouTube uses to uniquely identify the video.

**channel\_id** -> The ID that YouTube uses to uniquely identify the channel that the video was uploaded to.

**video-title** -> The video's title

**video\_description** -> The video's description.

**publish\_date** -> The date and time that the video was published. Note that this time might be different than the time that the video was uploaded. For example, if a video is uploaded as a private video and then made public at a later time, this property will specify the time that the video was made public.

**tags** -> A list of keyword tags associated with the video. Tags may contain spaces.

**category\_id** -> The YouTube video category associated with the video

**duration** -> The length of the video. The property value is an [ISO 8601](#) duration. For example, for a video that is at least one minute long and less than one hour long, the duration is in the format `PT#M#S`, in which the letters `PT` indicate that the value specifies a period of time, and the letters `M` and `S` refer to length in minutes and seconds, respectively. The `#` characters preceding the `M` and `S` letters are both integers that specify the number of minutes (or seconds) of the video. For example, a value of `PT15M33S` indicates that the video is 15 minutes and 33 seconds long.

**caption\_enabled** -> Indicates whether captions are available for the video.

**view\_count** -> The number of times the video has been viewed.

**like\_count** -> The number of users who have indicated that they liked the video.

**comment\_count** -> The number of comments for the video.



## Step 1: Read Video IDs

The previously created upload\_playlist.csv file was loaded, and the video\_id column was extracted. These video IDs serve as the input for the YouTube Videos API.

```
BASE_URL = "https://www.googleapis.com/youtube/v3"
```

```
upload_playlist_df = pd.read_csv('../upload_playlist.csv')
```

```
video_ids = upload_playlist_df[['video_id']]
```

```
video_ids.columns  
len(video_ids)
```

```
31358
```

## Step 2: Retrieve Video Metadata

The Videos API (videos.list) was called using the extracted video IDs. Since the API accepts a maximum of 50 video IDs per request, the IDs were processed in batches.

```
endpoint = "videos"  
videos_url = f"{BASE_URL}/{endpoint}"  
  
all_video_data = []  
  
for batch in range(0, len(video_ids), 50):  
    params = {  
        "key": API_KEY,  
        "part": "snippet,contentDetails,status,statistics",  
        "id": ','.join(video_ids.iloc[batch:batch+50]['video_id'])  
    }  
    response = requests.get(videos_url, params=params)  
    all_video_data.extend(response.json().get('items', []))
```

```
len(all_video_data)
```

```
31358
```

## Step 3: Remove Unprocessed Videos and Save

Some uploaded videos had not completed YouTube's processing and therefore contained incomplete metadata. This ensured that only fully processed videos with complete metadata were retained. After filtering out unprocessed videos, the DataFrame was exported.

```
# Removing data for video ids not having 'processed' status  
video_id_with_no_processed_status = []  
for data in all_video_data:  
    if data['status']['uploadStatus'] != 'processed':  
        video_id_with_no_processed_status.append(data['id'])  
video_id_with_no_processed_status
```

```
['iYt3c_X5bLs',  
'ySSRi5dfws4',  
'1dXIWxkLy3E',  
'gknWRtdelik',  
'5HkbMytJ5Uw',  
'q5Fa6sTo6W0',  
'J9mxt1XD_50',  
'iipR5yUp36o',  
'ZiChWM81Tho',  
'ucffZfKJ7gU',  
'jEK8Yw-SVPw',  
'Dhb21pSt8NM',  
'NUT_XmbUSSY',  
'XRb9c2XjZds']
```

```

videos_df = pd.DataFrame({
    "video_id": [data['id'] for data in all_video_data],
    "channel_id": [data['snippet']['channelId'] for data in all_video_data],
    "video-title": [data['snippet']['title'] for data in all_video_data],
    "video_description": [data['snippet']['description'] for data in all_video_data],
    "publish_date": [data['snippet']['publishedAt'] for data in all_video_data],
    "tags": [data['snippet'].get('tags') for data in all_video_data],
    "category_id": [data['snippet'].get('categoryId') for data in all_video_data],
    "duration": [data['contentDetails'].get('duration') for data in all_video_data],
    "dimension": [data['contentDetails'].get('dimension') for data in all_video_data],
    "caption_enabled" : [data['contentDetails'].get('caption') for data in all_video_data],
    "view_count": [data['statistics'].get('viewCount') for data in all_video_data],
    "like_count": [data['statistics'].get('likeCount') for data in all_video_data],
    "dislike_count": [data['statistics'].get('dislikeCount') for data in all_video_data],
    "favourite_count": [data['statistics'].get('favoriteCount') for data in all_video_data],
    "comment_count": [data['statistics'].get('commentCount') for data in all_video_data],
})

final_df = videos_df[~videos_df['video_id'].isin(video_id_with_no_processed_status)]

final_df.to_csv('videos.csv')

```

## STEPS INVOLVED FOR CREATION OF CATEGORY CSV FILE FROM API

**Endpoint** = <https://www.googleapis.com/youtube/v3/videoCategories>

**Columns:**

category\_id -> The ID that YouTube uses to uniquely identify the video category.

category\_name -> The video category's title.

### Step 1: Read Category IDs

The videos.csv dataset was loaded, and all unique category\_id values were extracted.

These category IDs represent the YouTube-assigned category for each video.

```

: videos_df = pd.read_csv('../videos.csv')

: len(videos_df)
category_ids = set(videos_df['category_id'])

: list(category_ids)

: [10, 20, 22, 23, 24, 25, 26, 27, 28]

```

### Step 2: Retrieve Category Details and Save

The videoCategories.list endpoint of the YouTube Data API was called using the extracted category IDs. This creates a lookup table that maps each category ID to its corresponding category name.

```
endpoint= "videoCategories"
category_url = f"{BASE_URL}/{endpoint}"
params= {
    "key":API_KEY,
    "part":'snippet',
    "id": category_ids,
}
```

```
response = requests.get(category_url,params=params)
```

```
channel_category_details=[]

for detail in response.json()['items']:
    channel_category_details.append(detail)
```

```
category_df = pd.DataFrame({
    "category_id" : [category['id'] for category in channel_category_details],
    "category_name" : [category['snippet']['title'] for category in channel_category_details]
})
```

```
category_df.to_csv('../category.csv')
```

## DATA CLEANING

The datasets collected from the YouTube Data API were examined for missing values, incorrect data types, inconsistent formats, and columns with little or no analytical value. Each issue was evaluated individually to determine whether cleaning was necessary and whether the issue would affect subsequent analysis.

### Step 1: Handling Date-like Video Titles

#### Dataset

videos

#### Column

video\_title

#### Issue

37 video titles resembled dates (for example, "12-08-2022").

#### Resolution

No changes were made.

#### Reason

These values represent legitimate video titles uploaded by creators and are not data quality errors.

### Step 2: Handling Blank Video Descriptions

#### Dataset

videos

### Column

video\_description

### Issue

2,141 videos have blank descriptions.

### Resolution

No cleaning was performed.

### Reason

Providing a description is optional on YouTube. Blank descriptions therefore represent genuine missing information rather than data collection errors.

## **Step 3: Handling Missing Video Tags**

### Dataset

videos

### Column

tags

### Issue

4,698 videos have no tags.

### Resolution

No cleaning was performed.

### Reason

Tags are optional metadata provided by creators.

## **Step 4: Converting Video Duration**

### Dataset

videos

### Column

duration

### Issue

The YouTube API returns duration using the ISO 8601 standard (e.g., PT4M23S) rather than a numeric duration.

### Resolution

The duration values were converted using:

```
isodate.parse_duration(x)
```

## DURATION COLUMN

```
videos_df['duration']
```

```
0      PT20S
1      PT44S
2      PT31S
3      PT15S
4      PT9S
...
31339   PT1H50M40S
31340   PT1H44M39S
31341     PT2H6M52S
31342     PT1H6M30S
31343   PT1H55M19S
Name: duration, Length: 31344, dtype: str
```

```
import isodate
videos_df["duration"] = videos_df["duration"].apply(
    lambda x: str(isodate.parse_duration(x))
)
```

```
videos_df['duration']
```

```
0      0:00:20
1      0:00:44
2      0:00:31
3      0:00:15
4      0:00:09
...
31339   1:50:40
31340   1:44:39
```

When the data was imported into Power BI, the duration appeared in the format:

0 days 00:04:23

Since all videos in the dataset had durations of less than one day, the "0 days" prefix was removed using the Replace Values transformation. The cleaned values (e.g., 00:04:23) were then converted from Text to the Duration data type.

Query Settings

PROPERTIES

Name

videos

APPLIED STEPS

Source

Promoted Headers1

Changed Type

Replaced Value

Changed Type1

Table.ReplaceValue("#Changed Type","0 days","",Replacer.ReplaceText,{"duration"})

|   | category_id | duration | caption_enabled | view_count | like_count | comment_count | view |
|---|-------------|----------|-----------------|------------|------------|---------------|------|
| 1 | 22          | 00:00:20 | FALSE           | 2017       | 13         | 2             |      |
| 2 | 22          | 00:00:44 | FALSE           | 10378      | 196        | 2             |      |
| 3 | 22          | 00:00:31 | FALSE           | 7591       | 176        | 5             |      |
| 4 | 22          | 00:00:15 | FALSE           | 11032      | 394        | 18            |      |
| 5 | 22          | 00:00:09 | FALSE           | 12159      | 248        | 3             |      |
| 6 | 22          | 00:00:15 | FALSE           | 5934       | 153        | 1             |      |

Query Settings

PROPERTIES

Name

videos

APPLIED STEPS

Source

Promoted Headers1

Changed Type

Replaced Value

Changed Type1

Table.TransformColumnTypes("#Replaced Value",{{"duration", type duration}})

|   | category_id | duration   | caption_enabled | view_count | like_count | comment_count | view |
|---|-------------|------------|-----------------|------------|------------|---------------|------|
| 1 | 22          | 0.00:00:20 | FALSE           | 2017       | 13         | 2             |      |
| 2 | 22          | 0.00:00:44 | FALSE           | 10378      | 196        | 2             |      |
| 3 | 22          | 0.00:00:31 | FALSE           | 7591       | 176        | 5             |      |
| 4 | 22          | 0.00:00:15 | FALSE           | 11032      | 394        | 18            |      |
| 5 | 22          | 0.00:00:09 | FALSE           | 12159      | 248        | 3             |      |
| 6 | 22          | 0.00:00:15 | FALSE           | 5934       | 153        | 1             |      |
| 7 | 22          | 0.00:00:12 | FALSE           | 2597       | 43         | 1             |      |
| 8 | 22          | 0.00:00:19 | FALSE           | 17404      | 241        | 15            |      |

### Reason

These transformations ensure that the duration is stored in a standard time format that supports sorting, filtering, aggregation, and duration-based calculations in Power BI, enabling accurate analysis and visualization of video lengths.

## **Step 5: Removing the Dimension Column**

### Dataset

videos

### Column

dimension

### Issue

Almost every record (31,343 out of 31,344) contains the value 2d.

### Resolution

The column was removed.

DROP COLUMNS

```
videos_df.drop(columns=['dimension','dislike_count','favourite_count'],inplace=True)
```

```
videos_df.columns
```

```
Index(['video_id', 'channel_id', 'video-title', 'video_description',  
      'publish_date', 'tags', 'category_id', 'duration', 'caption_enabled',  
      'view_count', 'like_count', 'comment_count'],  
      dtype='str')
```

### Reason

Since nearly every record contains the same value, the column provides no meaningful information for analysis.

## **Step 6: Validating Zero View Counts**

### Dataset

videos

### Column

view\_count

### Issue

Only 25 videos have zero views.

### Resolution

No changes were made.

### Reason

These values are likely genuine because the videos were recently uploaded or had not yet received any views.

## Step 7: Removing Dislike Count

### Dataset

videos

### Column

dislike\_count

### Issue

Every value is blank.

### Resolution

The column was removed.

### Reason

Since YouTube discontinued public dislike counts, the API no longer returns meaningful values.

DROP COLUMNS

```
videos_df.drop(columns=['dimension','dislike_count','favourite_count'],inplace=True)
```

```
videos_df.columns
```

```
Index(['video_id', 'channel_id', 'video-title', 'video_description',  
      'publish_date', 'tags', 'category_id', 'duration', 'caption_enabled',  
      'view_count', 'like_count', 'comment_count'],  
      dtype='str')
```

## Step 8: Handling Missing Like Counts

### Dataset

videos

### Column

like\_count

### Issue

187 videos have blank values.

### Resolution

No cleaning was performed.

### Reason

Creators can hide public like counts. During Power BI analysis, blank values are ignored while calculating average engagement metrics, preventing bias.

## Step 9: Removing Favourite Count

### Dataset

videos

### Column

favourite\_count

### Issue

Every record contains the value 0.

### Resolution

The column was removed.

### Reason

The YouTube API no longer uses this metric, making the column analytically useless.

DROP COLUMNS

```
videos_df.drop(columns=['dimension','dislike_count','favourite_count'],inplace=True)
```

```
videos_df.columns
```

```
Index(['video_id', 'channel_id', 'video-title', 'video_description',  
      'publish_date', 'tags', 'category_id', 'duration', 'caption_enabled',  
      'view_count', 'like_count', 'comment_count'],  
      dtype='str')
```

## Step 10: Handling Missing Comment Counts

### Dataset

videos

### Column

comment\_count

### Issue

569 videos have blank comment counts.

### Resolution

No cleaning was performed.

### Reason

Comment counts may be hidden by creators. Since blank values are ignored during average calculations in Power BI, they do not distort engagement metrics.



## Step 11: Handling Blank Channel Descriptions

### Dataset

channels

### Column

channel\_description

### Issue

86 channels have no description.

### Resolution

No cleaning was performed.

### Reason

Providing a channel description is optional.

## Step 12: Handling Missing Channel Country

### Dataset

channels

### Column

channel\_country

### Issue

170 channels have missing country information.

### Resolution

No cleaning or imputation was performed.

### Reason

The channel country is an important categorical attribute used in the analysis. Replacing missing values with arbitrary countries, the most frequent country, or any default value would introduce bias and produce misleading geographical insights. Since the missing values originate from channels that have not publicly specified their country, they represent genuine missing information rather than data collection errors. Therefore, the missing values were retained to preserve the integrity and authenticity of the dataset.

## Step 13: Validating Publish Date Against Channel Creation Date

### Datasets

videos

channels

### Issue

For approximately 50 records, the video publish date is earlier than the associated channel creation date.

### Resolution

No cleaning was performed.

### Reason

This behaviour is expected for YouTube Topic Channels. Topic channels are automatically generated by YouTube after music releases have already been published. Consequently, videos may legitimately have publication dates earlier than the creation date of the automatically generated Topic channel.

```
mask = (combined_df['publish_date'] - combined_df['channel_creation_date']).dt.days < 0
```

```
bad_data=combined_df[mask]
```

```
print(channels_df["channel_id"].duplicated().sum())
```

```
0
```

```
print(combined_df[["publish_date", "channel_creation_date"]].dtypes)
```

```
publish_date      datetime64[us, UTC]
channel_creation_date  datetime64[us, UTC]
dtype: object
```

```
bad_data['no_of_days'] = (combined_df['publish_date'] - combined_df['channel_creation_date']).dt.days
```

```
bad_data['channel_id'].unique()
```

```
<StringArray>
['UCKZn01AisL2kMV_2y04C9Fg']
Length: 1, dtype: str
```

```
len(bad_data)
```

```
50
```

Auto-generated YouTube Topic channels may have a channel\_creation\_date that is later than the publish\_date of videos associated with them. This occurs because Topic channels are created after music releases have already been published and are subsequently linked to the newly created channel.

So we will not remove these rows and consider them VALID

---

## Step 14: Correcting Data Types

Several columns required conversion to appropriate data types before analysis.

### Reason

Using the correct data types ensures accurate aggregation, filtering, sorting, visualization, and statistical analysis.

## Step 15: Validate Subscription Time Against Channel Creation Date

### Datasets

subscriptions

channels

### Issue

A validation was performed to check whether any subscription occurred before the corresponding channel was created.

## Resolution

No cleaning was performed.

## Reason

The subscription\_time and channel\_creation\_date fields were converted to the DateTime data type and validated after merging the subscriptions and channels datasets using channel\_id. No records were found where subscription\_time was earlier than channel\_creation\_date, confirming that all records satisfy the business rule and the dataset is consistent.

```
combined_df_1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   channel_id            108 non-null    str
1   subscriber_id         108 non-null    str
2   subscription_time      108 non-null    datetime64[us, UTC]
3   channel_title         108 non-null    str
4   channel_description    102 non-null    str
5   channel_creation_date  108 non-null    datetime64[us, UTC]
6   channel_country       105 non-null    str
7   thumbnail             108 non-null    str
8   subscriberCount       108 non-null    int64
9   totalViewCount        108 non-null    float64
10  totalVideoCount       108 non-null    int64
dtypes: datetime64[us, UTC](2), float64(1), int64(2), str(6)
memory usage: 9.4 KB
```

```
combined_df_1['channel_creation_date'] = pd.to_datetime(combined_df_1['channel_creation_date'],format='mixed')
```

```
mask = (combined_df_1['subscription_time'] - combined_df_1['channel_creation_date']).dt.days < 0
```

```
combined_df_1[mask==True]
```

| channel_id | subscriber_id | subscription_time | channel_title | channel_description | channel_creation_date | channel_country | thumbnail | subscriberCount | totalViewCount |
|------------|---------------|-------------------|---------------|---------------------|-----------------------|-----------------|-----------|-----------------|----------------|
|------------|---------------|-------------------|---------------|---------------------|-----------------------|-----------------|-----------|-----------------|----------------|

HENCE PROVED NO ROWS VIOLATING THE RULE (SUBSCRIPTION TIME SHOULD BE AFTER CHANNEL CREATION DATE)

## NEW COLUMNS/MEASURES CREATED

### MEASURES

#### 1) Average engagement

average engagement =

```
AVERAGEX(
    videos,
    DIVIDE(videos[comment_count]+videos[like_count],videos[view_count])
)
```

#### 2) Median description length

median description length = MEDIANX(videos, LEN(videos[video\_description]))

#### 3) Median duration minutes

median duration minutes =

```
MEDIANX(
    videos,
    HOUR(videos[duration]) * 60 + MINUTE(videos[duration])
    +SECOND(videos[duration])/ 60
)
```

#### 4) Median tag length

```
median tag length =  
MEDIANX(  
  videos,  
  VAR TagCount =  
    IF(  
      ISBLANK(videos[tags]) || videos[tags] = "",  
      0,  
      LEN(videos[tags])  
        - LEN(SUBSTITUTE(videos[tags], ",", ""))  
        + 1  
    )  
  RETURN TagCount  
)
```

## NEW COLUMNS

#### 1) Channel age

```
channel_age = DATEDIFF(channels[channel_creation_date],UTCTODAY(),YEAR)
```

#### 2) Videos per year

```
Videos per year = DIVIDE(channels[totalVideoCount],channels[channel_age])
```

#### 3) Duration minutes

```
duration minutes = HOUR(videos[duration]) * 60 + MINUTE(videos[duration]) +  
SECOND(videos[duration])/ 60
```

#### 4) Engagement

```
engagement = DIVIDE(videos[like_count]+videos[comment_count],videos[view_count])
```

#### 5) Publish Day of Week

```
publish day of week = FORMAT(videos[publish_date],"dddd")
```

#### 6) Video\_rank\_within\_category

```
Video_rank_within_category =  
VAR RankPercent =  
  DIVIDE(videos[view_ranking_within_category],  
  videos[total_videos_within_category])  
  
RETURN  
SWITCH(  
  TRUE(),  
  RankPercent <= 0.30, "Popular",  
  RankPercent <= 0.70, "Medium",  
  "Not Popular"  
)
```

#### 7) Video\_rank\_within\_channel

```

Video_rank_within_channel =
VAR RankPercent =
    DIVIDE(videos[view_ranking_within_channel],videos[total_videos_within_channel])

RETURN
SWITCH(
    TRUE(),
    RankPercent <= 0.30, "Popular",
    RankPercent <= 0.70, "Medium",
    "Not Popular"
)

```

## 8) View ranking within channel

```

: ranking_df = videos_df
: ranking_df['views_ranking'] = ranking_df.groupby(['channel_id'])['view_count'].rank(method='dense',ascending=False)

: ranking_df['views_ranking'] = ranking_df['views_ranking'].astype('Int64')

: ranking_df.head()

```

|   | video_id    | channel_id               | video-title                                       | video_description | publish_date              | tags | category_id | duration        | caption_enabled | view_count | like_coun |
|---|-------------|--------------------------|---------------------------------------------------|-------------------|---------------------------|------|-------------|-----------------|-----------------|------------|-----------|
| 0 | a_51N4XXSJM | UC6UTUmL5RbRPPRgnptAVu1g | New Bug? #brawlstars #brawler #bug                | NaN               | 2026-04-27 17:55:50+00:00 | NaN  | 22          | 0 days 00:00:20 | False           | 2017       | 13        |
| 1 | khu2dnYoMnI | UC6UTUmL5RbRPPRgnptAVu1g | Where is Leon? #brawlstars #brawl #supercell #... | NaN               | 2024-06-03 13:47:22+00:00 | NaN  | 22          | 0 days 00:00:44 | False           | 10378      | 196       |
| 2 | Qs0v10GRZFQ | UC6UTUmL5RbRPPRgnptAVu1g | Me 2020 fr #brawlstars #gaming #supercell #bra... | NaN               | 2024-06-02 17:35:34+00:00 | NaN  | 22          | 0 days 00:00:31 | False           | 7591       | 176       |
| 3 | KmPMNJWf_k  | UC6UTUmL5RbRPPRgnptAVu1g | I Hope Ash is Not Going away from Brawl Stars ... | NaN               | 2024-05-28 12:17:23+00:00 | NaN  | 22          | 0 days 00:00:15 | False           | 11032      | 394       |
| 4 | 69ci7Av6FDI | UC6UTUmL5RbRPPRgnptAVu1g | Check it, I DON'T LIKE THIS PIN #brawlstars #g... | NaN               | 2024-05-24 18:47:38+00:00 | NaN  | 22          | 0 days 00:00:09 | False           | 12159      | 248       |

## 9) Total videos within channel

```

: ranking_df['total_videos'] = (
:     ranking_df.groupby('channel_id')['video_id']
:     .transform('count')
: )

```

|       | video_id    | channel_id               | video-title                                       | video_description                                 | publish_date              | tags                                         | category_id | duration        | caption_ |
|-------|-------------|--------------------------|---------------------------------------------------|---------------------------------------------------|---------------------------|----------------------------------------------|-------------|-----------------|----------|
| 0     | a_51N4XXSJM | UC6UTUmL5RbRPPRgnptAVu1g | New Bug? #brawlstars #brawler #bug                |                                                   | 2026-04-27 17:55:50+00:00 | NaN                                          | 22          | 0 days 00:00:20 |          |
| 1     | khu2dnYoMnI | UC6UTUmL5RbRPPRgnptAVu1g | Where is Leon? #brawlstars #brawl #supercell #... |                                                   | 2024-06-03 13:47:22+00:00 | NaN                                          | 22          | 0 days 00:00:44 |          |
| 2     | Qs0v10GRZFQ | UC6UTUmL5RbRPPRgnptAVu1g | Me 2020 fr #brawlstars #gaming #supercell #bra... |                                                   | 2024-06-02 17:35:34+00:00 | NaN                                          | 22          | 0 days 00:00:31 |          |
| 3     | KmPMNJWf_k  | UC6UTUmL5RbRPPRgnptAVu1g | I Hope Ash is Not Going away from Brawl Stars ... |                                                   | 2024-05-28 12:17:23+00:00 | NaN                                          | 22          | 0 days 00:00:15 |          |
| 4     | 69ci7Av6FDI | UC6UTUmL5RbRPPRgnptAVu1g | Check it, I DON'T LIKE THIS PIN #brawlstars #g... |                                                   | 2024-05-24 18:47:38+00:00 | NaN                                          | 22          | 0 days 00:00:09 |          |
| ...   | ...         | ...                      | ...                                               | ...                                               | ...                       | ...                                          | ...         | ...             | ...      |
| 31339 | uQDYLEc3OqA | UCRVsqvXldgyPVS-a6fGa1IA | Shyne EXPOSES Diddy Fallout, Survived 10 Years... | 00:00 Seat Stolen Fallout\n01:13 From Rap to P... | 2026-04-09 16:01:27+00:00 | ['earn your leisure', 'business', 'finance', | 27          | 0 days 01:50:40 |          |

## 10) View ranking within category

```
videos_df['view_ranking_within_category']=videos_df.groupby('category_id')['view_count'].rank(method='dense',ascending=False)
```

```
videos_df['view_ranking_within_category']= videos_df['view_ranking_within_category'].astype('Int64')
```

```
videos_df.head()
```

| category_id | duration           | caption_enabled | view_count | like_count | comment_count | view_ranking_within_channel | total_videos_within_channel | view_ranking_within_category |
|-------------|--------------------|-----------------|------------|------------|---------------|-----------------------------|-----------------------------|------------------------------|
| 22          | 0 days<br>00:00:20 | False           | 2017       | 13.0       | 2.0           | 21                          | 26                          | 454                          |
| 22          | 0 days<br>00:00:44 | False           | 10378      | 196.0      | 2.0           | 9                           | 26                          | 351                          |
| 22          | 0 days<br>00:00:31 | False           | 7591       | 176.0      | 5.0           | 12                          | 26                          | 373                          |
| 22          | 0 days<br>00:00:15 | False           | 11032      | 394.0      | 18.0          | 6                           | 26                          | 345                          |
| 22          | 0 days<br>00:00:09 | False           | 12159      | 248.0      | 3.0           | 5                           | 26                          | 338                          |

## 11) Total videos within category

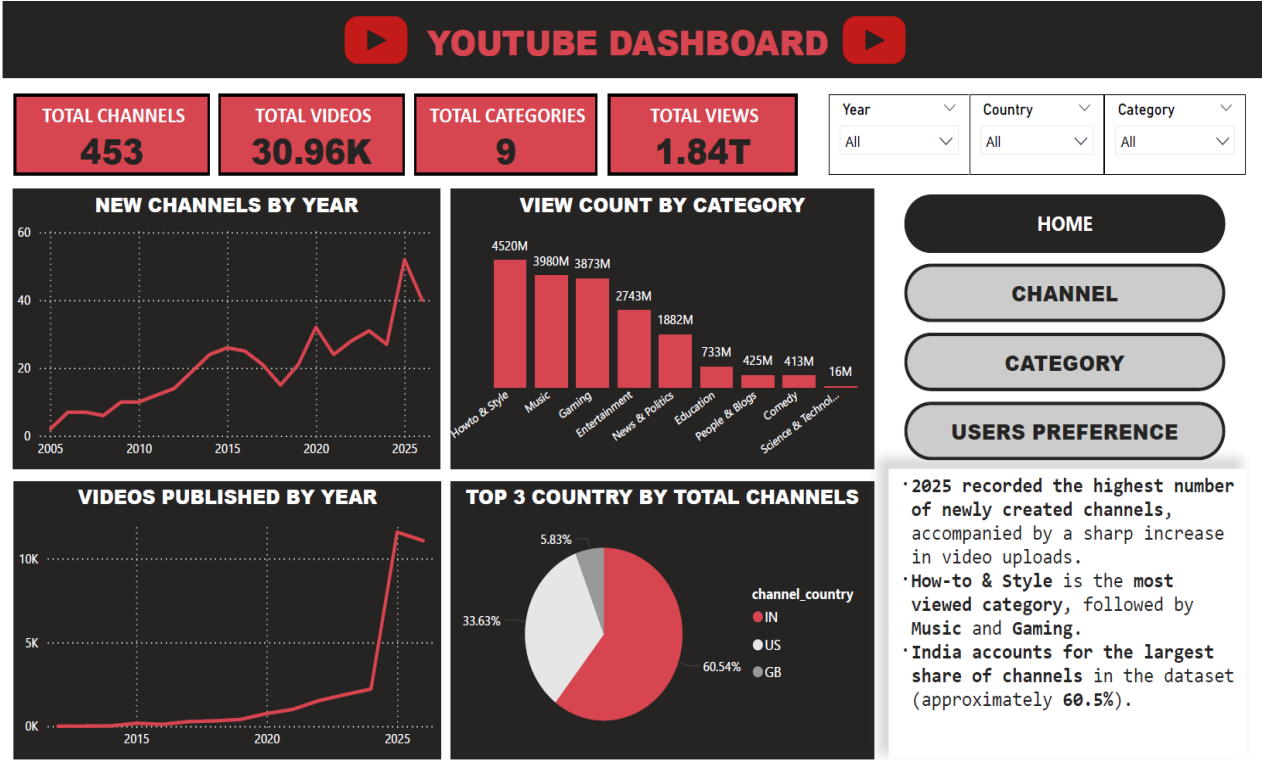
```
: category_counts=videos_df.groupby('category_id')['video_id'].count().reset_index(name='total_videos_within_category')
```

```
: videos_df = videos_df.merge(category_counts, how='left',on='category_id')
```

```
: len(videos_df)
```

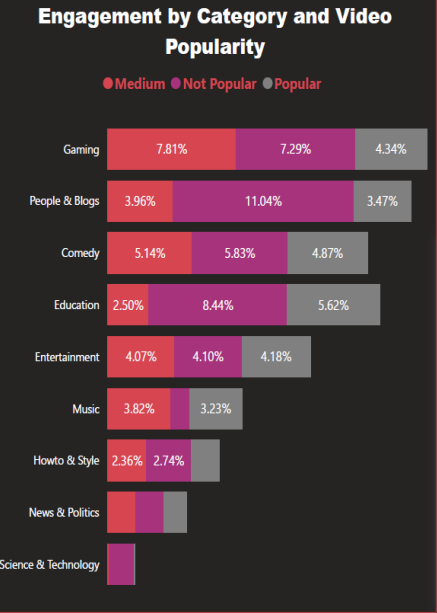
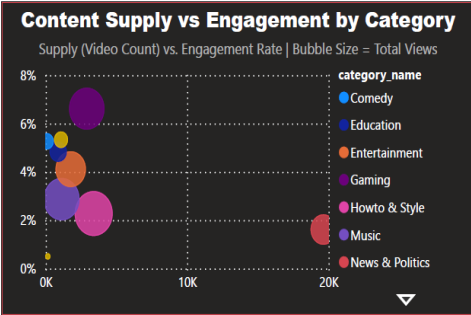
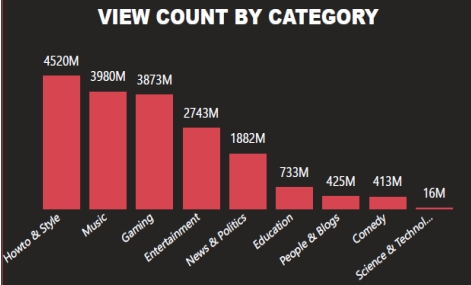
```
: 31344
```

DASHBOARD



YOUTUBE DASHBOARD

|                |              |                  |                |      |         |          |
|----------------|--------------|------------------|----------------|------|---------|----------|
| TOTAL CHANNELS | TOTAL VIDEOS | TOTAL CATEGORIES | AVG ENGAGEMENT | Year | Country | Category |
| 453            | 30.96K       | 9                | 2.56%          | All  | All     | All      |

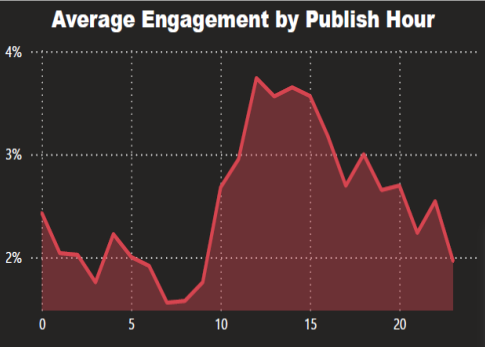
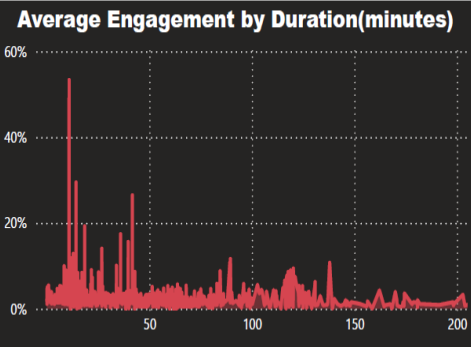


- HOME
- CHANNEL
- CATEGORY
- USERS PREFERENCE

- Howto & Style leads in total views but has one of the lowest engagement rates, indicating high reach but lower audience interaction.
- Gaming delivers the highest engagement despite a relatively smaller content supply.
- People & Blogs achieves strong engagement with comparatively fewer views and videos.
- News & Politics has high content volume but relatively low engagement, suggesting quantity alone does not drive audience interaction.
- Lower-viewed videos often achieve higher engagement within their categories, showing that views and engagement are not always correlated.
- Audience engagement depends more on content relevance than upload volume.

YOUTUBE DASHBOARD

|                |              |                  |             |      |         |          |
|----------------|--------------|------------------|-------------|------|---------|----------|
| TOTAL CHANNELS | TOTAL VIDEOS | TOTAL CATEGORIES | TOTAL VIEWS | Year | Country | Category |
| 453            | 30.96K       | 9                | 1.84T       | All  | All     | All      |



- HOME
- CHANNEL
- CATEGORY
- USERS PREFERENCE

- Videos under 45 minutes tend to generate higher engagement overall, while engagement generally declines for longer videos.
- Average engagement peaks between 11 AM and 3 PM. The difference between the average and median suggests the presence of a few highly engaged videos, but the higher median also indicates that engagement is generally stronger during this period.
- Engagement remains fairly consistent across the week, with a slight peak around Sunday-Monday.
- Tuesday and Thursday have the highest publishing activity, but this does not correspond to substantially higher engagement.
- New subscriptions have grown steadily over the years, with the highest additions in recent years.

